

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática · Escuela Profesional de Ingeniería de Software
Curso: Análisis y Diseño de Algoritmos (ADA) — Ciclo 2026-1

ESPECIFICACIÓN DEL PROYECTO GRUPAL — SEMANA 14

Sistema de Optimización de Portafolio con GA y DP

Implementación en Python · Despliegue en Streamlit · Datos de Yahoo Finance

Algoritmos integrados: Markowitz · NSGA-II · Programación Dinámica · Buy & Hold

Tickers default: FSM · VOLCABC1.LM · ABX.TO · BVN · BHP (mineras con operaciones en Perú)

Capital default: USD \$100,000 · Horizonte: 2015-01-01 a 2024-12-31

Prof. Mg. Ing. Ernesto D. Cancho-Rodríguez, MBA

MBA, The George Washington University (Washington, D.C., EE.UU.)

Lima, julio 2026

Contenido

Contenido	2
1. Introducción y Objetivos del Proyecto	3
1.1 Propósito	3
1.2 Conexión con los Temas 9–13 del curso	3
1.3 Qué se proporciona vs. qué deben hacer	3
2. Arquitectura del Sistema Streamlit	4
2.1 Estructura de archivos	4
2.2 Parámetros configurables en el sidebar	4
2.3 Elementos de interfaz obligatorios por módulo	4
Módulo 1 — Datos y Markowitz	4
Módulo 2 — NSGA-II Multiobjetivo	5
Módulo 3 — DP Rebalanceo	5
Módulo 4 — Comparación Cruzada	5
3. Descripción de los Notebooks Proporcionados	6
3.1 Módulo 1 — Datos y Markowitz	6
3.2 Módulo 2 — NSGA-II Multiobjetivo	6
3.3 Módulo 3 — DP Rebalanceo	6
3.4 Módulo 4 — Comparación Cruzada	7
4. Material de Referencia de Trabajos Anteriores	8
5. Diseño de Algoritmos Asistido por IA (AI-Assisted Algorithm Design)	9
5.1 Herramientas de IA generativa (chatbots)	9
5.2 Herramientas de codificación asistida por IA	9
5.3 Plataformas de vibe coding (generación completa de apps)	9
6. Prompts Hiper-Detallados para Conversión a Streamlit	10
6.1 Prompt — Homepage (app.py)	10
6.2 Prompt — Módulo 1 (Datos y Markowitz)	10
6.3 Prompt — Módulo 2 (NSGA-II)	11
6.4 Prompt — Módulo 3 (DP Rebalanceo)	11
6.5 Prompt — Módulo 4 (Comparación)	11
7. Instrucciones de Despliegue en Streamlit Community Cloud	13
8. Entregables Obligatorios	14
8.1 Estructura de carpetas en Google Drive	14
8.2 Estructura del Informe Word	14
9. Criterios de Evaluación	16

1. Introducción y Objetivos del Proyecto

1.1 Propósito

Este proyecto integrador de **Semana 14** consolida todos los algoritmos de optimización de portafolios trabajados durante el curso **Análisis y Diseño de Algoritmos (ADA)** en un **sistema web desplegable**. El equipo debe tomar los **4 notebooks proporcionados** (que ya contienen código funcional probado en Google Colab), entenderlos, y convertirlos en un **sistema Streamlit multimodular** desplegado en la nube, con interfaz interactiva, gráficos dinámicos y exportación de resultados.

1.2 Conexión con los Temas 9–13 del curso

- **Tema 9 (Greedy/GRASP):** Algoritmos voraces G2I y R2I para construir portafolios — superados por los métodos de este proyecto.
- **Tema 10 (Programación Dinámica):** Módulo 3 del sistema — backward induction de Bellman para rebalanceo óptimo con costos de transacción.
- **Tema 11 (B&B):** Branch & Bound para selección con cardinalidad — complementario al NSGA-II del Módulo 2.
- **Tema 12/13 (Algoritmos Genéticos):** Módulo 2 del sistema — NSGA-II multiobjetivo para generar la frontera Pareto retorno vs. riesgo.
- **Markowitz (media-varianza):** Módulo 1 del sistema — benchmark analítico con scipy.optimize.

1.3 Qué se proporciona vs. qué deben hacer

Se proporciona (ya resuelto)	El equipo debe hacer (trabajo de la semana)
Modulo1_Datos_Markowitz.ipynb — Notebook funcional con descarga de datos, frontera de Markowitz, simulación Buy&Hold y rebalanceo periódico.	Entender el notebook, ejecutarlo en Colab, y convertirlo en la página 1 del sistema Streamlit con widgets interactivos.
Modulo2_NSGA2_Multiobjetivo.ipynb — Notebook funcional con DEAP, frontera Pareto, 3 portafolios representativos, hypervolume, simulación GA con rebalanceo.	Entender el notebook, ejecutarlo en Colab, y convertirlo en la página 2 del sistema Streamlit con sliders para parámetros NSGA-II.
Modulo3_DP_Rebalanceo.ipynb — Notebook funcional con backward induction, 3 estrategias, heatmap tabla DP, evolución de riqueza.	Entender el notebook, ejecutarlo en Colab, y convertirlo en la página 3 del sistema Streamlit con sliders para λ_{TC} , T, paso de grilla.
Modulo4_Comparacion.ipynb — Notebook funcional con comparación cruzada de todos los métodos, tabla de métricas, gráficos superpuestos.	Entender el notebook y convertirlo en la página 4 (la más importante: comparación cruzada de GA vs DP vs Markowitz vs Buy&Hold).
Esta especificación con prompts hiper-detallados para la conversión a Streamlit.	Crear <code>app.py</code> + <code>pages/</code> , añadir <code>st.sidebar</code> , <code>st.metric</code> , <code>st.download_button</code> , desplegar en Streamlit Community Cloud, documentar en Word.

2. Arquitectura del Sistema Streamlit

2.1 Estructura de archivos

```

portfolio_optimizer/
├─ app.py                # Homepage con descripción del sistema
├─ pages/
│   ├─ 1_Datos_y_Markowitz.py    # Módulo 1
│   ├─ 2_NSGA2_Multiobjetivo.py  # Módulo 2
│   ├─ 3_DP_Rebalanceo.py        # Módulo 3
│   └─ 4_Comparacion.py          # Módulo 4
├─ utils/                  # Funciones compartidas (opcional)
├─ requirements.txt        # deap, yfinance, streamlit, plotly, openpyxl
└─ README.md              # Documentación del repo

```

2.2 Parámetros configurables en el sidebar

Parámetro	Widget Streamlit	Valor default	Descripción
Tickers	st.text_input	FSM, VOLCABC1.LM, ABX.TO, BVN, BHP	Lista de tickers separados por coma. El usuario puede cambiarla.
Fecha inicio	st.date_input	2015-01-01	Calendario desplegable para seleccionar fecha.
Fecha fin	st.date_input	2024-12-31	Calendario desplegable para seleccionar fecha.
Capital inicial (USD)	st.number_input	100,000	Textbox pre-rellenado con \$100,000.
NSGA-II: Población (MU)	st.slider	100	Rango: 50–300.
NSGA-II: Generaciones	st.slider	80	Rango: 30–200.
DP: Costo transacción (λ_{TC})	st.slider	0.001	Rango: 0.0001–0.01.
DP: Horizonte (periodos)	st.slider	12	Rango: 4–52.
Frecuencia rebalanceo	st.selectbox	Mensual	Opciones: Semanal, Mensual, Trimestral.
- Botón Cargar Datos YFinance - Botón Ejecutar Optimizador - Botón Descargar (para cada tabla importante): Ejemplo: Pesos del Portoflio Optimo, etc.	st.button	—	Ejecuta todos los modelos con los parámetros elegidos.

2.3 Elementos de interfaz obligatorios por módulo

Módulo 1 — Datos y Markowitz

- **Frontera eficiente de Markowitz** (scatter plot con acciones individuales + curva eficiente + punto máx Sharpe + punto mín varianza).
- **Pie chart** de composición del portafolio óptimo (máximo Sharpe).
- **Tarjetas st.metric()** con Sharpe Ratio, Sortino Ratio y volatilidad del portafolio óptimo.
- **Simulación de riqueza** Buy & Hold vs. Markowitz rebalanceado, con capital de \$100,000.
- **Botón de descarga** (st.download_button) para exportar pesos del portafolio a Excel (.xlsx).

Módulo 2 — NSGA-II Multiobjetivo

- **Frontera Pareto NSGA-II** superpuesta con la frontera de Markowitz, coloreada por Sharpe Ratio.
- **3 portafolios representativos** del frente Pareto: conservador, agresivo, máximo Sharpe. Con pie charts.
- **Evolución del hypervolume** por generación (gráfico de convergencia).
- **Simulación de riqueza** GA rebalanceado vs. Buy & Hold, con capital de \$100,000.
- **Botón de descarga** para exportar el frente Pareto completo y los 3 portafolios a Excel.

Módulo 3 — DP Rebalanceo

- **Evolución de riqueza** de 3 estrategias: Buy & Hold vs. DP Optimizado vs. Siempre Rebalanceado.
- **Heatmap de la tabla DP** (costos óptimos por estado y periodo).
- **Timeline de rebalanceos** (en qué periodos se rebalanceó según la política DP).
- **Tarjetas st.metric()** con riqueza final, Sharpe y costos acumulados de cada estrategia.
- **Botón de descarga** para exportar la tabla de simulación a Excel.

Módulo 4 — Comparación Cruzada

- **Gráfico de barras comparativo** de métricas (Sharpe, riqueza final, max drawdown) para todos los métodos.
- **Gráfico de evolución de riqueza** superpuesto de TODOS los métodos en un solo plot.
- **Tabla resumen** con todas las métricas por método, visible como st.dataframe() y descargable como Excel.
- **Ranking** automático de métodos por Sharpe Ratio y por riqueza final.

3. Descripción de los Notebooks Proporcionados

Se proporcionan **4 notebooks funcionales** como archivos adjuntos a esta especificación. Cada notebook corre directamente en **Google Colab** sin modificaciones. Los nombres de archivo son:

- **Modulo1_Datos_Markowitz.ipynb**
- **Modulo2_NSGA2_Multiobjetivo.ipynb**
- **Modulo3_DP_Rebalanceo.ipynb**
- **Modulo4_Comparacion.ipynb**

3.1 Módulo 1 — Datos y Markowitz

Inspirado en: Notebook D del Equipo 3 (Eq3_T12_GA_Notebook_D_Preparacion_Datos.ipynb) y INICIO.ipynb del proyecto de investigación doctoral del profesor.

Referencia bibliográfica: Markowitz, H. (1952). "Portfolio Selection". *Journal of Finance*, 7(1), 77-91.
<https://doi.org/10.2307/2975974>

Qué hace: Descarga precios de cierre de 5 tickers mineros (FSM, VOLCABC1.LM, ABX.TO, BVN, BHP) desde Yahoo Finance para el periodo 2015-2024. Calcula retornos logarítmicos diarios, vector de retornos esperados anualizados (μ), y matriz de covarianza anualizada (Σ). Resuelve los portafolios de máximo Sharpe y mínima varianza con `scipy.optimize.minimize (SLSQP)`. Genera la frontera eficiente con 200 puntos. Simula la evolución de riqueza (\$100,000) para Buy&Hold y para Markowitz rebalanceado mensualmente.

3.2 Módulo 2 — NSGA-II Multiobjetivo

Inspirado en: Notebook C del Equipo 3 (Eq3_T12_GA_Notebook_C_NSGA2_Multiobjetivo.ipynb) y Notebook C del Equipo 5 (Eq5_T13_GA_Notebook_C_NSGA_II_Multiobjetivo.ipynb).

Referencia bibliográfica: Deb, K., Pratap, A., Agarwal, S. & Meyarivan, T. (2002). "A fast and elitist multiobjective genetic algorithm: NSGA-II". *IEEE Trans. Evolutionary Computation*, 6(2), 182-197.
<https://doi.org/10.1109/4235.996017>

Qué hace: Implementa el Algoritmo NSGA-II con DEAP para optimización bi-objetivo (maximizar retorno, minimizar riesgo) del portafolio. Genera el frente de Pareto y lo superpone con la frontera de Markowitz. Extrae 3 portafolios representativos (conservador, agresivo, máx Sharpe). Calcula hypervolume e IGD. Simula riqueza con portafolio GA rebalanceado.

3.3 Módulo 3 — DP Rebalanceo

Inspirado en: Notebook B del Equipo 5 (Eq5_T10_DP_NotebookB_Algoritmo_DP.ipynb) y Notebook DP del Equipo 1 (Eq1_T10_DP_Algoritmo_de_Programación_Dinámica.ipynb, que reproduce el paper de Vaezi et al.).

Referencia bibliográfica: Vaezi Jezeie, F. et al. (2022). "Constrained portfolio optimization with discrete variables: An algorithmic method based on dynamic programming". PLoS ONE, 17(8), e0271811.

<https://doi.org/10.1371/journal.pone.0271811>

Qué hace: Implementa backward induction de la ecuación de Bellman para rebalanceo multiperiodo. Discretiza el espacio de estados (pesos del portafolio). Compara 3 estrategias: Buy&Hold, DP Optimizado y Siempre Rebalanceado. Produce heatmap de la tabla DP, gráfico de evolución de riqueza y métricas comparativas.

3.4 Módulo 4 — Comparación Cruzada

Nuevo (creado para este proyecto): Integra los resultados de los 3 módulos anteriores en una comparación unificada. Superpone las curvas de evolución de riqueza de TODOS los métodos, genera tabla resumen de métricas, y produce un ranking automático.

4. Material de Referencia de Trabajos Anteriores

Los siguientes archivos de trabajos de equipos de semanas anteriores se comparten como **referencia complementaria**. Estos notebooks sirvieron como materia prima para construir los 4 notebooks proporcionados:

#	Archivo	Tema / Equipo
1	Eq3_T12_GA_Notebook_D_Preparacion_Datos.ipynb	GA — Equipo 3 — Datos + Markowitz
2	Eq3_T12_GA_Notebook_C_NSGA2_Multiobjetivo.ipynb	GA — Equipo 3 — NSGA-II
3	Eq5_T13_GA_Notebook_C_NSGA_II_Multiobjetivo.ipynb	GA — Equipo 5 — NSGA-II
4	Eq5_T10_DP_NotebookB_Algoritmo_DP.ipynb	DP — Equipo 5 — Backward induction
5	Eq5_T10_DP_NotebookC_Analisis_Sensibilidad.ipynb	DP — Equipo 5 — Sensibilidad
6	Eq1_T10_DP_Algoritmo_de_Programación_Dinámica.ipynb	DP — Equipo 1 — Vaezi et al. 2022
7	INICIO.ipynb	Investigación doctoral — Config tickers mineros

5. Diseño de Algoritmos Asistido por IA (AI-Assisted Algorithm Design)

Se autoriza y se incentiva el uso de **herramientas de IA generativa** para la conversión de notebooks a Streamlit, la generación de código, y la depuración. El equipo **debe documentar su proceso de uso de IA** en el informe: qué herramienta usó, qué prompt le dio, qué obtuvo y qué modificó manualmente.

5.1 Herramientas de IA generativa (chatbots)

- **Claude AI (Anthropic):** <https://claude.ai/> — Modelos: Claude Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 4.6. Contexto 1M tokens.
- **ChatGPT (OpenAI):** <https://chatgpt.com/> — Modelos: GPT-5.5, GPT-5.5 Instant, GPT-5.4 Thinking.
- **Gemini (Google):** <https://gemini.google.com/> — Modelos: Gemini 3.5 Flash, Gemini 3.1 Pro.
- **DeepSeek AI:** <https://chat.deepseek.com/> — Modelos: DeepSeek-R1, DeepSeek-V3.
- **Grok (xAI):** <https://x.com/i/grok> — Modelos: Grok 4.3, Grok 4.20 Reasoning.

5.2 Herramientas de codificación asistida por IA

- **Claude Code (Anthropic):** <https://docs.anthropic.com/en/docs/claude-code> — Codificación agéntica CLI.
- **GitHub Copilot:** <https://github.com/features/copilot> — Autocompletado en VS Code. Gratuito para estudiantes.
- **Replit AI:** <https://replit.com/> — Desarrollo + despliegue en la nube con agente IA.
- **Cursor IDE:** <https://www.cursor.com/> — VS Code con IA nativa integrada.
- **Windsurf (Codeium):** <https://codeium.com/windsurf> — Autocompletado y chat con IA.
- **Google AI Studio:** <https://aistudio.google.com/> — Prototipado con Gemini.

5.3 Plataformas de vibe coding (generación completa de apps)

- **Manus AI:** <https://manus.im/app> — Agente autónomo que genera y despliega apps.
- **Rocket.new:** <https://www.rocket.new/> — Apps completas desde un prompt.
- **Firebase Studio (Google):** <https://studio.firebase.google.com/> — IDE cloud con Gemini.
- **Lovable:** <https://lovable.dev/es> — Apps React desde lenguaje natural.

6. Prompts Hiper-Detallados para Conversión a Streamlit

Los siguientes prompts basados en **etiquetas XML** están diseñados para ser copiados directamente en **Claude AI, ChatGPT o Gemini** para generar el código Streamlit de cada módulo. El equipo puede usarlos como punto de partida y adaptarlos.

6.1 Prompt — Homepage (app.py)

<contexto>

Eres un desarrollador Python senior especializado en Streamlit y finanzas cuantitativas.

Estás construyendo la homepage de un Sistema de Optimización de Portafolio que integra 3 métodos: Markowitz (media-varianza), NSGA-II (algoritmo genético multiobjetivo con DEAP), y Programación Dinámica (backward induction de Bellman). Los datos vienen de Yahoo Finance. El capital default es USD \$100,000. Los tickers default son 5 mineras: FSM, VOLCABC1.LM, ABX.TO, BVN, BHP.

</contexto>

< tarea>

Genera un archivo app.py para Streamlit con: título del sistema, descripción de los 4 módulos (Datos/Markowitz, NSGA-II, DP, Comparación), st.sidebar con: text_input para tickers (default: 'FSM, VOLCABC1.LM, ABX.TO, BVN, BHP'), dos date_input con calendarios (default: 2015-01-01 a 2024-12-31), number_input para capital (default: 100000), y un botón 'Ejecutar Análisis'. Incluir st.session_state para compartir parámetros entre páginas.

</ tarea>

<restricciones>

- Python 3.10+, Streamlit 1.x.
- Paleta: fondo blanco, primario azul oscuro (#1F3864), acento granate (#800000), dorado (#C5961A).
- Fuente: Calibri o sans-serif. Diseño limpio, profesional.
- El sidebar debe ser visible en todas las páginas.
- Incluir disclaimer: 'Los datos son simulaciones con fines académicos y no constituyen asesoría de inversión.'

</restricciones>

6.2 Prompt — Módulo 1 (Datos y Markowitz)

<contexto>

Tienes un notebook de Google Colab funcional (Modulo1_Datos_Markowitz.ipynb) que descarga datos de Yahoo Finance, calcula retornos logarítmicos, resuelve el portafolio de Markowitz (máx Sharpe y mín varianza con `scipy.optimize`), genera la frontera eficiente con 200 puntos, y simula la evolución de riqueza para Buy&Hold vs. Markowitz rebalanceado mensualmente con capital de \$100,000.

</contexto>

<tarea>

Convierte este notebook en un archivo `pages/1_Datos_y_Markowitz.py` para Streamlit. Los parámetros (`tickers`, `fechas`, `capital`) vienen de `st.session_state` (configurados en el sidebar del homepage). Usa `st.pyplot()` para la frontera eficiente, `plotly.express.pie()` para el pie chart del portafolio, `st.metric()` en 3 columnas para Sharpe/Sortino/Volatilidad, y `st.download_button()` para exportar los pesos a Excel con `openpyxl`. Incluir spinner con `st.spinner('Optimizando portafolio...')` durante el cálculo.

</tarea>

6.3 Prompt — Módulo 2 (NSGA-II)

<contexto>

Tienes un notebook (Modulo2_NSQA2_Multiobjetivo.ipynb) que implementa NSGA-II con DEAP para portafolio bi-objetivo. Genera frontera Pareto vs. Markowitz, hypervolume, 3 portafolios representativos, y simulación de riqueza GA rebalanceado.

</contexto>

<tarea>

Convierte en `pages/2_NSQA2_Multiobjetivo.py`. Añadir sliders para MU (50-300, default 100) y NGEN (30-200, default 80). Mostrar barra de progreso `st.progress()` durante la evolución. Gráfico Pareto con plotly (interactivo, hover muestra pesos). Pie charts para los 3 portafolios con `st.columns(3)`. Descarga Excel del frente Pareto completo.

</tarea>

6.4 Prompt — Módulo 3 (DP Rebalanceo)

<contexto>

Tienes un notebook (Modulo3_DP_Rebalanceo.ipynb) con backward induction de Bellman, 3 estrategias (Buy&Hold, DP, Siempre-rebalanceado), heatmap de tabla DP, evolución de riqueza con \$100,000.

</contexto>

<tarea>

Convierte en `pages/3_DP_Rebalanceo.py`. Sliders para λ_{TC} (0.0001-0.01), T (4-52), paso de grilla (0.02-0.20). `st.metric()` para riqueza final de cada estrategia. Heatmap con `plotly.imshow()`. Gráfico de evolución con 3 curvas coloreadas. Descarga Excel de la simulación.

</tarea>

6.5 Prompt — Módulo 4 (Comparación)

<contexto>

Tienes un notebook (Modulo4_Comparacion.ipynb) que compara Markowitz, GA (NSGA-II), DP y Buy&Hold: tabla de métricas, gráfico de barras, gráfico de riqueza superpuesto.

</contexto>

<tarea>

Convierte en pages/4_Comparacion.py. Tabla interactiva con `st.dataframe()`. Gráfico de barras con `plotly`. Gráfico de riqueza superpuesto con `plotly` (interactivo). Ranking automático. Botón para descargar reporte completo en Excel con múltiples hojas (una por método).

</tarea>

7. Instrucciones de Despliegue en Streamlit Community Cloud

1. **Crear un repositorio público en GitHub** con la estructura de archivos descrita en la sección 2.1.
2. **Subir todos los archivos** (app.py, pages/, requirements.txt, README.md) al repo.
3. Ir a <https://share.streamlit.io/> y conectar la cuenta de GitHub.
4. **Seleccionar el repo, la branch (main) y el archivo principal (app.py).**
5. **Hacer clic en Deploy.** Streamlit construye el entorno e instala las dependencias.
6. **Obtener la URL pública** (formato: <https://xxxxx.streamlit.app/>) e incluirla en el informe Word.

requirements.txt mínimo:

```
streamlit>=1.30
yfinance>=0.2
deap>=1.4
plotly>=5.0
openpyxl>=3.1
scipy>=1.10
numpy>=1.24
pandas>=2.0
matplotlib>=3.7
```

8. Entregables Obligatorios

#	Entregable	Formato	Dónde subir
1	Notebooks ejecutados en Google Colab (4 módulos)	.ipynb (ejecutados)	GitHub (repo público) + Google Drive (EqX.W14)
2	Sistema Streamlit desplegado y funcionando	URL pública .streamlit.app	GitHub (repo público para despliegue)
3	Código fuente del sistema Streamlit (app.py + pages/)	.py + requirements.txt	GitHub (repo público) + Google Drive (EqX.W14)
4	Informe de documentación del sistema	.docx	Google Drive (EqX.W14)
5	Diagrama de Componentes UML (PlantUML) de la arquitectura	.wsd + .png	Insertado en el Informe Word + Google Drive

8.1 Estructura de carpetas en Google Drive

```
EqX.W14 Proyecto Optimizacion GA DP/
├── Informe_Proyecto_W14.docx
├── notebooks/
│   ├── Modulo1_Datos_Markowitz.ipynb
│   ├── Modulo2_NSGA2_Multiobjetivo.ipynb
│   ├── Modulo3_DP_Rebalanceo.ipynb
│   └── Modulo4_Comparacion.ipynb
├── streamlit/
│   ├── app.py
│   ├── pages/ (4 archivos .py)
│   └── requirements.txt
├── diagramas/
│   ├── componentes.wsd
│   └── componentes.png
├── URL_sistema_desplegado.txt
└── Link_a_Video_Expo_Demo_sistema_desplegado_YT.txt
```

8.2 Estructura del Informe Word

7. **Carátula** (universidad, curso, título, integrantes, fecha, URLs del repo y del sistema desplegado).
8. **Introducción** (descripción del sistema, objetivos, conexión con el curso ADA).
9. **Descripción de los módulos** (qué hace cada módulo, qué algoritmo implementa, capturas de pantalla).
10. **Proceso de uso de IA** (qué herramientas usaron, qué prompts, qué obtuvieron, qué modificaron).
11. **Resultados** (capturas del sistema desplegado, métricas financieras, comparación de métodos).
12. **Diagrama de Componentes UML** (PlantUML, con protocolo BATD, insertado y explicado).
13. **Conclusiones** (¿qué método optimiza mejor? ¿el rebalanceo mejora el rendimiento? ¿GA supera a Markowitz?).
14. **Referencias** (papers, artículos, URLs de herramientas).

9. Criterios de Evaluación

#	Criterio	Peso	Descripción
1	Notebooks ejecutados en Colab	15%	Los 4 notebooks corren sin errores, producen gráficos y métricas correctas.
2	Sistema Streamlit funcional	30%	Los 4 módulos funcionan como páginas web, sidebar interactivo, gráficos visibles, botones de descarga.
3	Despliegue en Streamlit Cloud	15%	URL pública funcionando, repo GitHub público, requirements.txt correcto.
4	Módulo de comparación cruzada	15%	Gráficos superpuestos, tabla de métricas, ranking de métodos, exportación Excel.
5	Informe Word + Diagrama UML	15%	Documentación del sistema, capturas, proceso de IA, diagrama de componentes PlantUML.
6	VIDEO	10%	Expo pregrabada de demo y explicación del código